



American Express – Default Prediction

Predicting if a customer will default in the future

COMP9417 - Project
Term 2, 2022

Team Overfitting

zIDs: Joe Doyle - z5180548, Malin Thiele - z5182862, John Lim - z5334623

August 1, 2022

1 Introduction

For businesses, which lend money to customers, credit default prediction is important for managing risk. Such predictions allow for the optimisation of the decision of whether to lend to customers or not. American Express (AMEX) is the world’s largest payment card company. AMEX released a competition on Kaggle to predict the probability that a customer does not pay back their credit card balance amount based on their monthly consumer profile[1]. As part of the competition, an industrial sized data set was provided. This data, which includes training and testing datasets, includes historical and anonymous customer profile information. AMEX also released a specific evaluation metric as part of the competition (hereinafter referred to as AMEX Metric). The start and ending date for the competition was the 25th May 2022 and 24th August 2022, respectively. [1]

The purpose of this report was to provide the methodology along with the results and discussion that was used to attempt to maximise the AMEX Metric using the training dataset provided. The report was structured as follows; Section 2 provided a description of the datasets provided by AMEX. Section 3 provided a brief overview and explanation of the AMEX Metric. Section 4 provided the pre-processing and feature engineering methodology. Section 5 and 6 provided an overview of the model selection and optimisation methodology. Section 7 presented the optimization results and discussion. Section 8 provided the AMEX scores for each model. Section 9 provided the conclusion.

2 Dataset

2.1 Description

There were 3 files provided, which were `train_data.csv`, `train_labels.csv` and `test_data.csv`. `train_data.csv` provided the training data which included multiple monthly customer profile information statements (varying between 1 to 13 per customer). Customers were identified based on a *customer_id*. There were 458,913 total customers and 5,531,451 total statements provided. `train_labels.csv` provided the target binary variable for each *customer_id* in `train_data.csv`. The *target* variable, representing whether a default event occurred (1) or not (0), was determined based on whether the customer paid back their due amount in 120 days following their final statement [2]. There were 340,085 (~74% of total customers) examples of *target* = 0 and 118,828 (~26% of total customers) examples of *target* = 1. Thus, there was an unequal number of examples given for each class provided in the training data. `test_data.csv` provided the test data which was to be used to predict the *target* variable for each *customer_id* provided. There were 924,621 customers and 11,363,762 total statements (varying between 1 to 13 per customer) provided. Using `test_data.csv`, a submission file with the probability of future default (*target* = 1), for each respective *customer_id* was provided to Kaggle in order to evaluate performance.

2.2 Customer Features

Each customer statement had 188 anonymized and normalized features which were split up into 5 categories. These categories were: delinquency, spend, payment, balance, and risk. An overview of the number of features per category is given in Table 1 below. All features were numerical with the exception of D_63, D_64, D_66, D_68, D_114, D_116, D_117, D_120, D_126, B_30 and B_38, which were

Table 1: Breakdown of Customer Feature Categories

Feature Category	Number of Features
Delinquency (D_*)	96
Spend (S_*)	21
Payment (P_*)	3
Balance (B_*)	40
Risk (R_*)	28

As shown above, delinquency features made up over 50% of the total features, followed by balance features, which made up ~21%. Payment features were the least provided feature type.

3 Evaluation Metric

y_pred was the predicted probabilities of future default for each customer. y_true was the binary values indicating whether a default actually occurred for each customer. The AMEX metric was an equally weighted average of the Normalised Gini Coefficient (G) and the percentage of customer defaults identified in the top 4% of y_pred values (D). A weight of 20 is given to non-defaulting y labels ($y = 0$) to adjust for downsampling.

G was calculated as follows. y_pred and y_true values were concatenated in a panda's dataframe and then sorted in descending order based on y_pred . A weight (w) column was added to the dataframe where the row value was 20 if $y_true = 0$ or 1 if $y_true = 1$. A percentage weight cumulative sum (pwc) column was then added which was calculated by taking the individual weight in w on each row and dividing it by the total sum of w and then taking the cumulative sum down the dataframe. A cumulative positive (cp) column was then added which took the cumulative count down the dataframe where $y_true = 1$. G was then calculated by taking the values in cp and dividing them by the total number of customers where $y_true = 1$ minus the values in pwc all multiplied by w for each individual row in the dataframe and then summing them. G was subsequently normalized by dividing by the same calculated value as before but where $y_pred = y_true$.

D was calculated as follows. y_pred and y_true values were concatenated in a panda's dataframe and then sorted in descending order based on y_pred . A weight (w) column was added to the dataframe where the row value was 20 if $y_true = 0$ or 1 if $y_true = 1$. A 4% cut-off was then calculated as 4% of the total sum of column w . A weight cumulative sum (wc) column was then added which was calculated by taking the cumulative sum of w down the dataframe. D was then calculated as the number of customers where $y_true = 1$ and wc was less than the 4% cut-off all divided by the total number of customers where $y_true = 1$.

The AMEX metric was then calculated as $0.5(G + D)$. Clearly, predicting high probability of default for defaulting customers is most valuable based on the calculation of G and D . The code, an example provided by AMEX, is given in Appendix A.

4 Pre-processing and Feature Engineering

4.1 Pre-processing

Categorical features within the training and test sets, which were described in Section 2, were one-hot encoded. One-hot encoding (OHE) involves converting unique categorical values within a categorical feature into its own feature and assigning a binary value, either 1 or 0, to it [3]. This was undertaken since some models which were used throughout this study (refer to Section 5) required inputs which were numeric, such as logistic regression. Models including decision tree ensembles do not require such numeric inputs, but for the sake of consistency, such models were trained on data where one-hot encoded had been undertaken as a pre-processing step.

Approximately 10% of values within the training and test datasets were missing. All data was retained by use of imputation, whereby empty values were filled using the feature mean. The mean was used to retain the mean of the normalized feature. A drawback of this method was that the mean is considered a reasonable estimate for a randomly selected observation from a normal distribution, but if missing values were not strictly random some inconsistent bias may be present [13]. All training and test datasets (originally `train_data.csv` and `test_data.csv`) were then converted to Apache parquet files to improve the speed at which data could be read into Python. Parquet files are noted as being 34 times faster than csv files for querying [5]. After pre-processing, training/test data contained 222 features compared to an original 188.

4.2 Feature Engineering

Since the binary target variable was calculated based on whether a customer did not pay back their due amount 120 days following their most recent statement, it was thought that most information would be contained within the latest customer statement provided [1]. Thus, a base feature set was defined (hereafter named Feature Set 1) which was the last (i.e., most recent) statement for each *customer_id* using the original 222 features after pre-processing. There was thus 1 example per *customer_id*, reduced from a value between 1 to 13 inclusive.

To include historical customer information, new features were created and considered for selection to be appended

onto Feature Set 1. There were between 1 and 13 statements per customer (See Section 2). New features that were considered included the mean, min, max, standard deviation, sum, median, first, last minus first, last divided by first and the log of each of these for each customer for each of the original 222 features for the last, last 3, last 6 and all statements. It was thought that calculating such values using a different number of historical statements could capture historical customer information not otherwise contained in Feature Set 1. Furthermore, transformation of certain features allowed for greater separability between defaulters and non-defaulters, whereby log transformations were particularly effective in treating skewed continuous features. An additional 16,056 features were thus considered. An overview is below.

Table 2: New Features Created to be Evaluated for Selection

Features	Calculation	Period
All 222 Features in training and test set after pre-processing	mean, min, max, standard deviation (std), sum, median, first, last minus first, last divided by first, log mean, log min, log max, log std, log median, log first, log last minus first, log last divided by first	Last statement, last 3 statements, last 6 statements and all statements

Using the new features given in Table 2, a 2nd feature set (hereafter named Feature Set 2) was created. This was done by taking the absolute value of the Pearson Correlation Coefficient between each new created feature and the target variable [9]. The Pearson Correlation Coefficient measures the linear correlation between the created feature and the target [9]. While we acknowledge that correlation does not necessarily imply causation, due to the large number of newly created features considered, we wanted to consider features where there was at least some evidence of a positive or negative relationship between a newly created feature and the target. For each of the 222 features in Feature Set 1, the highest absolute value Pearson Correlation Coefficient was obtained from using all calculations and periods given in Table 2. The results of this analysis were contained within `correlation_features.txt` within the `feature_engineering` directory. The highest valued newly created features for each of the 222 features were then appended onto Feature Set 1 if they were not already contained within Feature Set 1. 207 additional features were added. The list of all features in Feature Set 1 and 2 can be found in `Feature_Set_Features.txt` within the `data_preprocessing` directory. Using Feature Set 2, optimal feature sets were found for each of the classification models tested in this study. Such methodologies were explained in Section 6.

5 Model Selection

Ensemble modelling was used in this classification task as it reduces the generalization error of the prediction[7]. This method is commonly used in practical applications of machine learning. The four types of independent models used in the ensemble were XGBoost’s `XGBClassifier`, LightGBM’s `LGBMClassifier`, SkLearn’s `RandomForestClassifier` and a Logistic Regression model built using `cuML`. All models were subsequently stacked using a hyperparameter optimized `cuML` Logistic Regression where each feature was the predictions given by the base models.

Decision trees make up the majority portion of the ensemble. These types of models were chosen due to their non-parametric nature which allows them to learn high dimensional data and provide good accuracy. They are very good at ignoring features that have little prediction value and are also very efficient to train [11].

Random forest is one of the ensemble models used, it fits a number of different decision tree classifiers (*n_estimators*) using the bagging technique (aggregated bootstrap) and random feature subsampling. This is used to create independence in the individual decision trees to help reduce variance. Subsamples of the dataset with varying rows and features will be used to train the trees. The bootstrap parameter will be set to true. Gini impurity will be used to measure the quality of a node split. The feature used to perform a node split will be randomly permuted, so splits produced may be different. For sklearn random forest, predictions are done by averaging the predictions of the ensembled trees.

Logistic regression is the linear model chosen; it uses the sigmoid function to produce a prediction probability of $[0, 1]$ for a defaulting customer. It is a simple model capable of achieving good results if the dataset is linearly separable and makes no assumption on the classes’ distributions. Feature importance can be done by evaluating its coefficients. It uses gradient descent to minimize its cross entropy loss (log loss) function. To reduce overfitting, L1 or L2 regularization will be used and the C inverse regularization strength parameter will be tuned for this. With

the L1 penalty term added to the loss, it is capable of doing feature elimination, this helps remove features with zero coefficients[4].

6 Optimisation Methodology

6.1 Optimal Feature Selection

6.1.1 Lasso Regularization

Feature selection is performed using the logistic regression classifier with L1 regularization. Nested cross validation is performed. The outer cross validation is used to evaluate the performance of the models. Features will be eliminated based on its frequency of elimination in the inner cross validated models. When the coefficients (*coef_* attribute) of the L1 classifier goes to zero, the feature will be eliminated. An inner cross validation will be performed with a grid search of C parameters ranging from [0.00002, 0.2] with 40 evenly spaced samples. The parameter C is the inverse regularization strength of the L1 classifier, smaller value means larger penalty on features with larger coefficients. The average count and frequency of features eliminated will be calculated based on each C parameter. The number of features eliminated will be obtained from the averages of the outer folds with the best C parameter that produces the best accuracy. The final set of selected features will then be produced by eliminating the most frequently eliminated features.

6.1.2 RFECV (Recursive Feature Elimination - Cross Validated)

Cross validated recursive feature elimination (RFECV) was the methodology used to obtain the optimal feature set for each individual tree ensemble. 4-fold stratified cross validation was used to reduce unwanted bias in the results and maintain an even distribution of classes. RFECV recursively removed poorly predicting features one-by-one from Feature Set 2 based on feature importance, which was defined as gain, with an objective to maximise the AMEX metric. Each time a feature was removed, a model was trained using the remaining features. The cross-validated scores of these models using the AMEX metric were used to determine the optimal subset of features for each individual model. This method of feature selection was chosen due to the high accuracy of the results even though the speed is slower than some of the alternatives[10]. The `sklearn.feature_selection.RFECV` implementation was used throughout this study with *importance_type* defined as "gain".

6.2 Hyperparameter Optimisation

Automated hyperparameter tuning was performed using the optuna package. Some of the reasons we chose to use this package are the ability to set a fixed length for the optimisation, visualizations of parameter importance, integrates well with Pandas DataFrames, and the fact that optuna (unlike methods such as sklearn's GridSearchCV) uses bayesian optimization to make intelligent decisions on which hyperparameters to test in upcoming trials. All the Optuna studies used 4-fold stratified cross validation in the objective function where the model was trained to reduce the bias and variance of the predictions. The sampler in our study used a TPE (Tree-structured Parzen Estimator) algorithm to choose the next set of hyperparameters. 100 trials were run for each ensemble model, 200 trials were run for the logistic regression cuML model. cuML is a python library which mirrors sklearn machine learning algorithms and uses the GPU. Due to the amount of data and features, feature selection and model training will be performed using the cuML library instead of sklearn, this has helped speed up the process from days to hours. The tables below provide an overview of the hyperparameters and their tuning ranges.

Table 3: Logistic Regression[12]

Hyperparameter	Description	Range
<i>penalty</i>	The norm used for the penalty term. L1 for lasso regression and L2 for ridge regression. Used to penalize models with high coefficients. L1 can be used for feature elimination as it will zero out coefficients.	{L1, L2}
<i>C</i>	The inverse regularization strength. Must be a positive float value. Lower value signifies higher penalty. Reduces overfitting by favoring models with lower complexity and lower coefficients.	[0.00002, 0.2]
<i>max_iter</i>	The maximum iterations allowed for the solvers to converge. Higher iterations may increase accuracy.	[1000, 2000]

<i>linesearch_max_iter</i>	The maximum linesearch optimization iterations for the solvers. Linesearch is done per outer iterations, used to search the optimal magnitude to move along the search direction which minimizes the loss.	[70, 200]
----------------------------	--	-----------

Table 4: XGBClassifier[15]

Hyperparameter	Description	Range
<i>n_estimators</i>	This parameter is the number of boosting rounds. The higher this value, the more trees will be built. Since each tree learns off the residuals of the previous one, high values can lead to overfitting.	[100, 1200]
<i>booster</i>	This parameter refers to the type of learner used in the model. Dart is similar to gbtrees however it includes a drop out feature.	{gbtree, dart}
<i>reg_lambda</i>	This is the model’s L2 regularisation parameter which is used to penalise leaf values to promote tree sparsity.	[1e-8, 1]
<i>reg_alpha</i>	This is the model’s L1 regularisation parameter which is used to penalise leaf values to promote tree sparsity.	[1e-8, 1]
<i>max_depth</i>	This is the maximum tree depth for each base learner. High values of this parameter can cause overfitting.	[1, 9]
<i>learning_rate</i>	This parameter is the shrinkage that happens every boosting round. This parameter will make the boosting process more or less conservative. There is a relationship between learning rate and number of boosting rounds.	[1e-8, 1]
<i>gamma</i>	This parameter determines the minimum loss reduction required to make another split at a leaf of the tree. Larger values of gamma will promote more conservative algorithms.	[1e-8, 1]
<i>grow_policy</i>	This parameter determines the tree growth policy, depthwise promotes splitting at the leaves, while lossguide promotes splitting at the point that will give the highest loss reduction.	{depthwise, lossguide}

Table 5: LGBMClassifier[8]

Hyperparameter	Description	Range
<i>n_estimators</i>	This parameter is the number of boosting rounds. The higher this value, the more trees will be built. Since each tree learns off the residuals of the previous one, high values can lead to overfitting.	[600, 1300]
<i>num_leaves</i>	This parameter controls the maximum number of leaves for each base learner. Larger number of leaves can lead to overfitting.	[5, 200]
<i>lambda_1</i>	This is the model’s L1 regularisation parameter which is used to penalise leaf values to promote tree sparsity.	[1e-8, 60]
<i>lambda_2</i>	This is the model’s L1 regularisation parameter which is used to penalise leaf values to promote tree sparsity.	[1e-8, 60]
<i>feature_fraction</i>	This parameter randomly selects features to include for training in each boosting round. This is used to avoid overfitting and speed up training.	[0.1, 1]
<i>learning_rate</i>	This parameter is the boosting learning rate.	[0.01, 0.1]
<i>bagging_fraction</i>	This parameter randomly selects data to include for training in each boosting round. This is used to avoid overfitting and speed up training. This is similar to <i>feature_fraction</i> but on part of the data instead of features.	[0.1, 1]
<i>bagging_frequency</i>	This parameter determines how frequently bagging is applied. A value of 0 means there will be no bagging and a value of n means bagging will be performed every nth iteration.	[1, 6]
<i>min_child_samples</i>	This parameter determines the minimal amount of data in one leaf. This is used to prevent overfitting in trees that grow leafwise.	[10, 250]

Table 6: RandomForestClassifier[14]

Hyperparameter	Description	Range
----------------	-------------	-------

<i>n_estimators</i>	The total number for decision trees built for the forest. Number of ensemble trees; uses averaging to provide predictions. More estimators can reduce variance.	[100, 1200]
<i>max_depth</i>	The maximum depth of each decision tree. Higher depth may increase overfitting; as all leaves can be pure. Can split until all leaves have less than the <i>min_samples_split</i> .	[10, 100]
<i>min_samples_split</i>	The minimum number of samples needed to split an internal node.	[2, 10]
<i>min_samples_leaf</i>	The minimum number of samples in each leaf node. Note if an internal node creates a split which is lesser than <i>min_samples_split</i> , the split will not be allowed.	[1, 5]

For stacking, a cuML Logistic Regression model was used to ensemble the individual models. The stacking model used the training data predictions from the base learners as features. The base learners were trained using 5-fold stratified cross validation. The hyperparameters of the stacking model were tuned using Optuna with 5-fold stratified cross validation and the hyperparameter ranges are given in Table 7.

Table 7: Logistic Regression Stacking Model Optuna Tuning Range

C	Penalty	max_iter	linesearch_max_iter
[0.00002, 0.2]	{L1, L2}	[1000, 2000]	[70, 200]

7 Optimisation Results and Discussion

The below sections present the output of optimal feature selection and hyperparameter tuning for each of the models in section 5. The output of the Optuna optimizations have been visually shown in Appendix B using parallel coordinate plots and hyperparameter importance plots. The code to produce such plots are provided in the optimisation directory and the results of the optimization is located in the results directory of the project submission.

7.1 Logistic Regression

Plot below shows the average of all 5 outer folds of the nested cross validation with regularization strength C, test accuracy and number of features eliminated. As the inverse of regularization strength increases, the model accuracy increases, however, the number of features eliminated decreases. This makes sense as the penalty is weaker, the model doesn't penalize larger valued coefficients, thus, more non-zeroed coefficients. We will pick the optimal point with an accuracy of 0.781 and 100 features eliminated, the regularization strength is approximately 0.02.

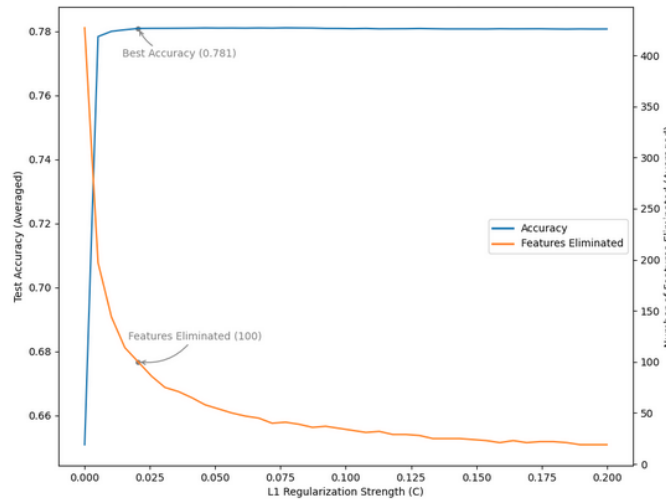


Figure 1: Feature selection with Logistic Regression L1 (Averaged over 5 folds)

It was found that 62% of the features eliminated are delinquency. The parameter to be tuned for this logistic regression model will be C (inverse regularization strength), penalty type (L1 or L2), *max_iter* (the maximum iteration allowed to converge), *linesearch_max_iter* (the maximum number of linesearch iterations for the solvers). Using Optuna on 200 trials to maximize cross validated objectives, it managed to achieve an accuracy of 0.7798 after searching for the optimal parameters.

Table 8: Logistic Regression Hyperparameter Importances

C	Penalty	max_iter	linesearch_max_iter
0.98	0.02	0.0	0.0

The table above shows that the regularization strength C is the most important hyperparameter. L1 is also the preferred penalty type used for the logistic regression model. The table below shows the optimal hyperparameters returned by optuna.

Table 9: Logistic Regression Hyperparameter Importances

C	Penalty	max_iter	linesearch_max_iter
0.03233	L1	1463	161

7.2 XGBClassifier

The plot below shows the maximum cross validation score achieved for a sub-sample of features of a given size. It can be seen that the maximum AMEX score using this model is able to be achieved using 174 features. The figure shows that using additional features added no improvement to the AMEX score. These 174 features were defined as the optimal features using this model. Thus, 255 features were able to be removed from Feature Set 2 in order to obtain the optimal feature set, using XGBClassifier.

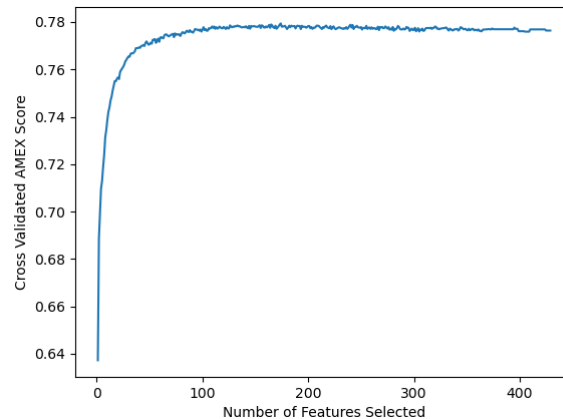


Figure 2: Maximum Cross Validated AMEX Score vs. Number of Selected Features for XGBClassifier

The features included in the optimal feature set are in XGB_feature_ranking.csv. The table below shows the importances of the top 4 hyperparameters when tuning the XGBClassifier model to maximize the AMEX score.

Table 10: XGBClassifier Hyperparameter Importances (Top 4)

learning_rate	max_depth	n_estimators	grow_policy
0.71	0.21	0.05	0.02

This clearly demonstrates that the learning rate had the largest effect on the overall prediction ability of the XGBClassifier model. The max depth of the trees and number of boosting rounds also had a significant effect on the performance. The regularisation parameters alpha, lambda and gamma have practically no effect on the model performance. The final obtained hyperparameters are given in table below.

Table 11: XGBClassifier Optimal Hyperparameters

n_estimators	booster	reg_lambda	reg_alpha	max_depth	learning_rate	gamma	grow_policy
988	gbtree	9.5786e-08	0.0002776	6	0.0002776	5.9977e-06	lossguide

7.3 LGBMClassifier

The plot below shows the maximum cross validation score achieved for a sub-sample of features of a given size. It can be seen that the maximum AMEX score using this model is able to be achieved using 140 features. The figure shows that using additional features added no improvement to the AMEX score. These 140 features were defined as the optimal features using this model. Thus, 289 features were able to be removed from Feature Set 2 in order to obtain the optimal feature set, using LGBMClassifier. Interestingly, it can be noticed that LGBMClassifier is able to obtain a 0.003 higher AMEX Score compared to XGBClassifier with 34 less features.

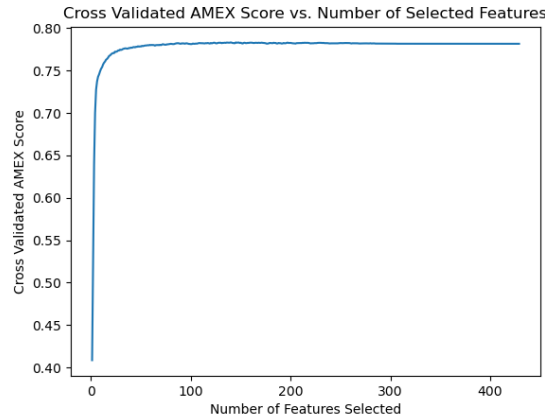


Figure 3: Maximum Cross Validated AMEX Score vs. Number of Selected Features for LGBMClassifier

The features included in the optimal feature set are in LightGBM_feature_ranking.csv. The table below shows the importances of the top 4 hyperparameters when tuning the LGBMClassifier model to maximize the AMEX score.

Table 12: LGBMClassifier Hyperparameter Importances (Top 4)

learning_rate	bagging_fraction	lambda_l1	num_leaves
0.79	0.14	0.02	0.02

This clearly demonstrates that the learning rate had the largest effect on the overall prediction ability of the LGBMClassifier model. The bagging fraction also had a significant effect on the performance. Lambda L1, number of leaves and number of boosting rounds all had a minimal effect on the model performance. The final obtained hyperparameters are given in table below.

Table 13: LGBMClassifier Optimal Hyperparameters

n_estimators	num_leaves	learning_rate	lambda_l1	lambda_l2
871	160	0.01773	9.9411e-08	10.87526
feature_fraction	bagging_fraction	bagging_freq	min_child_samples	
0.5428	0.8422	5	228	

7.4 RandomForestClassifier

The plot below shows the maximum cross validation score achieved for a sub-sample of features of a given size. It can be seen that the maximum AMEX score using this model is able to be achieved using 85 features. The figure shows that using additional features added no improvement to the AMEX score. These 85 features were defined as the optimal features using this model. Thus, 344 features were able to be removed from Feature Set 2 in order to obtain the optimal feature set, using Random Forest Classifier.

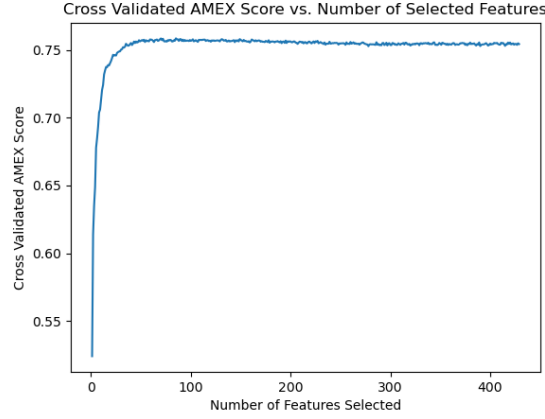


Figure 4: Maximum Cross Validated AMEX Score vs. Number of Selected Features for RandomForestClassifier

The features included in the optimal feature set are in `RandomForest_feature_ranking.csv`. The table shows the importances of the hyperparameters when tuning the `RandomForestClassifier` model to maximize the AMEX score.

Table 14: RandomForestClassifier Hyperparameter Importances

<code>n_estimators</code>	<code>max_depth</code>	<code>min_samples_split</code>	<code>min_samples_leaf</code>
0.70	0.20	0.08	0.02

The most important parameter to be tuned for the Random Forest Classifier is the `n_estimators`, this determines how many decision trees are built. This makes sense since higher amount of ensembled decision tree can reduce variance. After a search from [2, 1200], it has found a value of `n_estimators` of about 927. Max depth is the second important hyperparameter. It can be seen that `min_samples_leaf` and `min_samples_split` have minimal effect on model performance.

Table 15: RandomForestClassifier Optimal Hyperparameters

<code>n_estimators</code>	<code>max_depth</code>	<code>min_samples_split</code>	<code>min_samples_leaf</code>
927	77	8	4

7.5 Stacking

The optimal hyperparameters identified is given in the table below.

Table 16: Logistic Regression Stacking Model Optuna Tuning Range

<code>C</code>	<code>Penalty</code>	<code>max_iter</code>	<code>linesearch_max_iter</code>
0.0003026	L1	1456	114

8 Model Results

The table below shows the 5-fold stratified cross validated AMEX score using the training dataset and the corresponding Kaggle score achieved through predictions on the test set. It can be seen that there's a little difference in the cross validated score and the Kaggle score.

Table 17: Logistic Regression Model Results

Model	Features	Parameters	Cross Validated Score	Kaggle Score
Logistic Regression	Feature Set 1	Default	0.774	0.772
	Optimized	Default	0.779	0.779
	Optimized	Optimized	0.779	0.779
XGBClassifier	Feature Set 1	Default	0.762	0.775
	Optimized	Default	0.767	0.779
	Optimized	Optimized	0.786	0.788
LGBMClassifier	Feature Set 1	Default	0.777	0.779

	Optimized	Default	0.781	0.782
	Optimized	Optimized	0.789	0.789
RandomForestClassifier	Feature Set 1	Default	0.760	0.761
	Optimized	Default	0.762	0.763
	Optimized	Optimized	0.768	0.769
Stacking Model	Base Model Predictions	Optimized	0.791	0.791

Across all individual models, using feature selection improved both the cross validated and Kaggle scores. Optimizing the hyperparameters improved every single model except for Logistic Regression in which it had no effect. Stacking all individual models improved both the cross validated and Kaggle score by 0.002. This is not a significant increase when compared to the highest score observed by individual models (LGBMClassifier optimized). A possible reason that there was such a marginal increase in the score when stacking the models is that there was a high correlation in the individual base models, such correlation can be seen in Appendix C. We have found that the stacked model weighted the models in the order LGBMClassifier, XGBClassifier, Linear Regression and the least weighted was RandomForestClassifier.

9 Conclusion

In conclusion, we implemented four different machine learning models. The stacking of these models did not provide a significant improvement on the prediction of the best base model. This was due to high correlation of the base models' test predictions. Stacking results could possibly be improved by including all the models that may provide less correlated predictions to the other models, these may include Naive Bayes and Neural Networks. Feature engineering provided us with the largest improvements in the AMEX score, we would have liked to further analyze the relationships between individual features if we had more time. At the time of writing this, the 31st of July 2022, we are 2319th on the Kaggle public leaderboard with a score of 0.791. The top score is currently 0.801.

References

- [1] American Express. American express - default prediction. <https://www.kaggle.com/competitions/amex-default-prediction/overview>, May 2022.
- [2] American Express. American express - default prediction. <https://www.kaggle.com/competitions/amex-default-prediction/data>, May 2022.
- [3] Amanda Fawcett. One hot encoding. <https://www.educative.io/blog/one-hot-encoding#what>, Feb 2021.
- [4] GeeksforGeeks. Advantages and disadvantages of logistic regression. <https://www.geeksforgeeks.org/advantages-and-disadvantages-of-logistic-regression/>, Sep 2020.
- [5] Butterfly Thoughts in Performance. Probably the best alternative to csv storage: Parquet data. <https://geekflare.com/parquet-csv-data-storage/>, Nov 2021.
- [6] kaggle. Amex competition metric (python). <https://www.kaggle.com/code/inversion/amex-competition-metric-python>, May 2022.
- [7] Vijay Kotu and Bala Deshpande. Chapter 2 - data science process. In Vijay Kotu and Bala Deshpande, editors, *Data Science (Second Edition)*, pages 19–37. Morgan Kaufmann, second edition edition, 2019.
- [8] lightgbm. lightgbm.lgbmclassifier. <https://lightgbm.readthedocs.io/en/latest/pythonapi/lightgbm.LGBMClassifier.html>, July 2022.
- [9] Gianluca Malato. How to measure feature importance in a binary classification model. <https://medium.com/data-science-reporter/how-to-measure-feature-importance-in-a-binary-classification-model-d284b8c9a301>, Mar 2019.
- [10] Adi Mustaqim, Sumarni Adi, Yoga Pristyanto, and Yuli Astuti. The effect of recursive feature elimination with cross-validation (rfecv) feature selection algorithm toward classifier performance on credit card fraud detection. pages 270–275, June 2021.
- [11] PythonProgrammingLanguage. Advantages of a decision tree for classification. <https://pythonprogramminglanguage.com/what-are-the-advantages-of-using-a-decision-tree-for-classification/>.
- [12] rapids. Logistic regression. <https://docs.rapids.ai/api/cuml/stable/api.html#logistic-regression>, July 2022.
- [13] Baijanyanta Roy. All about missing data handling. <https://towardsdatascience.com/all-about-missing-data-handling-b94b8b5d2184>, Sep 2019.
- [14] sklearn. sklearn.ensemble.randomforestclassifier. [learn.org/stable/modules/generated/sklearn.ensemble.RandomForestClassifier.html](https://sklearn.org/stable/modules/generated/sklearn.ensemble.RandomForestClassifier.html), July 2022.
- [15] xgboost. Xgboost-python api reference. https://xgboost.readthedocs.io/en/stable/python/python_api.html, July 2022.

Appendix - A: AMEX Metric Calculation

```
In [1]: import numpy as np
import pandas as pd
from pathlib import Path

input_path = Path('/kaggle/input/amex-default-prediction/')

Amex Metric
This is a python version of the metric for the Amex competition. Additional details can be found on the competition Evaluation page.

In [2]: def amex_metric(y_true: pd.DataFrame, y_pred: pd.DataFrame) -> float:

    def top_four_percent_captured(y_true: pd.DataFrame, y_pred: pd.DataFrame) -> float:
        df = (pd.concat([y_true, y_pred], axis='columns')
              .sort_values('prediction', ascending=False))
        df['weight'] = df['target'].apply(lambda x: 20 if x==0 else 1)
        four_pct_cutoff = int(0.04 * df['weight'].sum())
        df['weight_cumsum'] = df['weight'].cumsum()
        df_cutoff = df.loc[df['weight_cumsum'] <= four_pct_cutoff]
        return (df_cutoff['target'] == 1).sum() / (df['target'] == 1).sum()

    def weighted_gini(y_true: pd.DataFrame, y_pred: pd.DataFrame) -> float:
        df = (pd.concat([y_true, y_pred], axis='columns')
              .sort_values('prediction', ascending=False))
        df['weight'] = df['target'].apply(lambda x: 20 if x==0 else 1)
        df['random'] = (df['weight'] / df['weight'].sum()).cumsum()
        total_pos = (df['target'] * df['weight']).sum()
        df['cum_pos_found'] = (df['target'] * df['weight']).cumsum()
        df['lorenz'] = df['cum_pos_found'] / total_pos
        df['gini'] = (df['lorenz'] - df['random']) * df['weight']
        return df['gini'].sum()

    def normalized_weighted_gini(y_true: pd.DataFrame, y_pred: pd.DataFrame) -> float:
        y_true_pred = y_true.rename(columns={'target': 'prediction'})
        return weighted_gini(y_true, y_pred) / weighted_gini(y_true, y_true_pred)

    g = normalized_weighted_gini(y_true, y_pred)
    d = top_four_percent_captured(y_true, y_pred)

    return 0.5 * (g + d)

Simple Benchmark
We can create a simple benchmark using the average of the feature P_2 for each customer.

In [3]: train_data = pd.read_csv(
    input_path / 'train_data.csv',
    index_col='customer_ID',
    usecols=['customer_ID', 'P_2'])

train_labels = pd.read_csv(input_path / 'train_labels.csv', index_col='customer_ID')

In [4]: ave_p2 = (train_data
    .groupby('customer_ID')
    .mean()
    .rename(columns={'P_2': 'prediction'}))

# Scale the mean P_2 by the max value and take the complement
ave_p2['prediction'] = 1.0 - (ave_p2['prediction'] / ave_p2['prediction'].max())

In [5]: print(amex_metric(train_labels, ave_p2)) # 0.572773

0.5727729219888231
```

Figure 5: AMEX metric Calculation[?]

Appendix - B: Hyperparameter Optimisation Visualisations

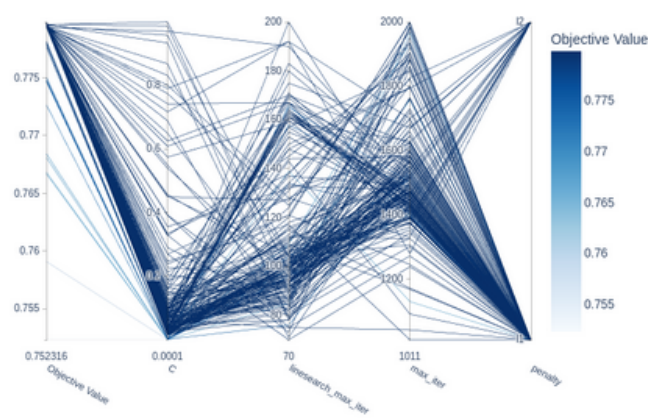


Figure 6: Logistic Regression Parallel Coordinate Plot

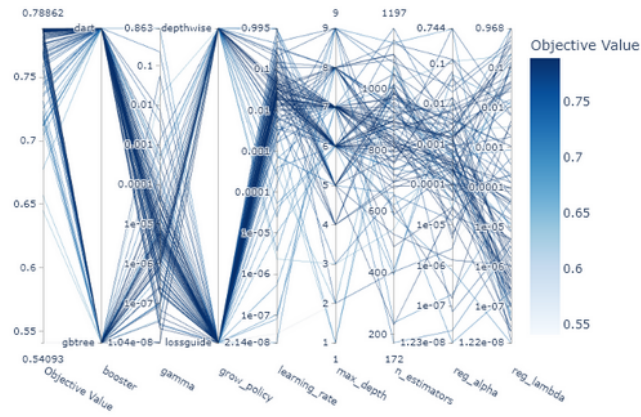


Figure 7: XGBClassifier Parallel Coordinate Plot

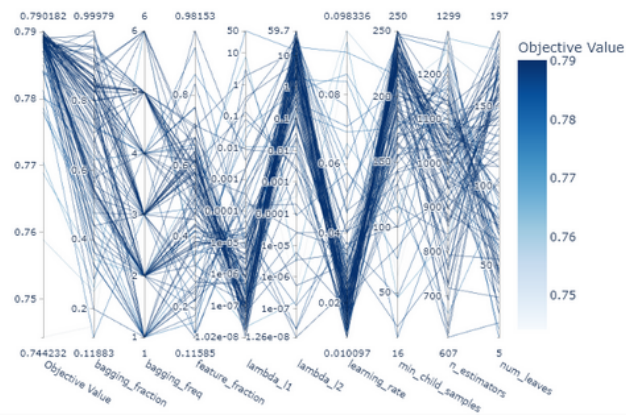


Figure 8: LGBMClassifier Parallel Coordinate Plot

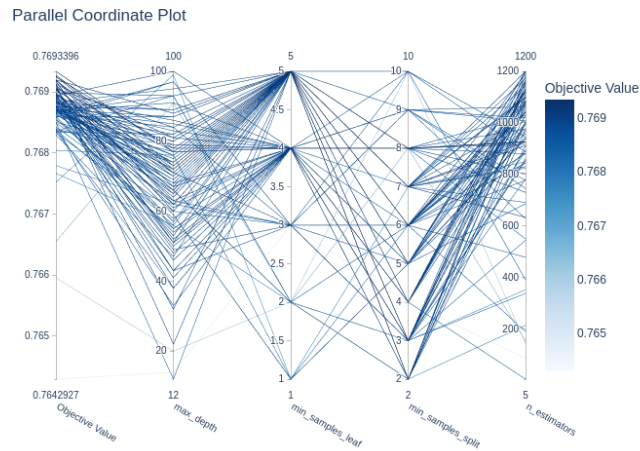


Figure 9: Random Forest Parallel Coordinate Plot

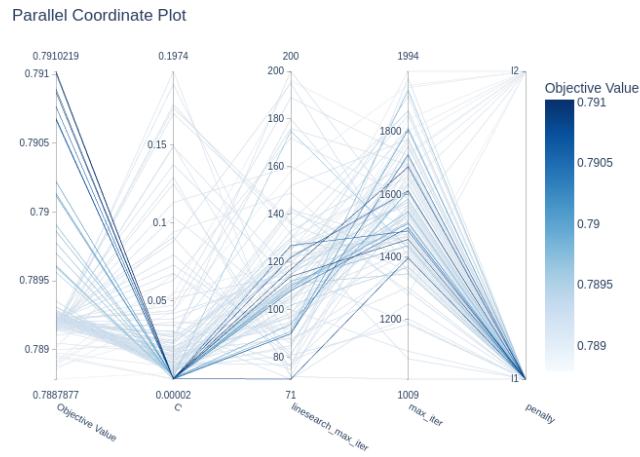


Figure 10: Stacking Model Parallel Coordinate Plot

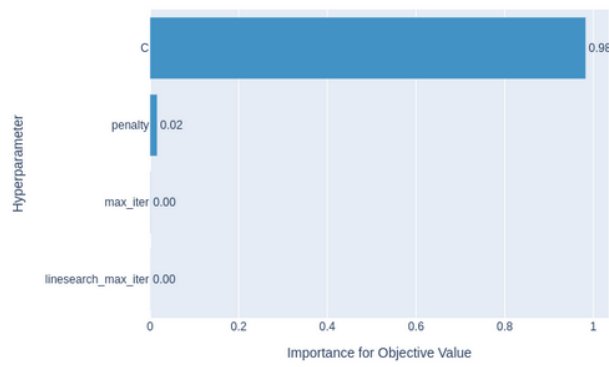


Figure 11: Logistic Regression Hyperparameter Importances

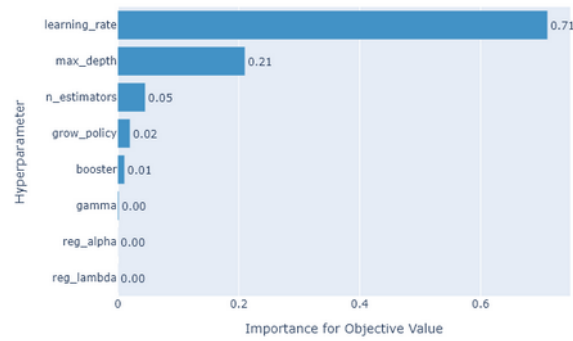


Figure 12: XGBClassifier Hyperparameter Importances

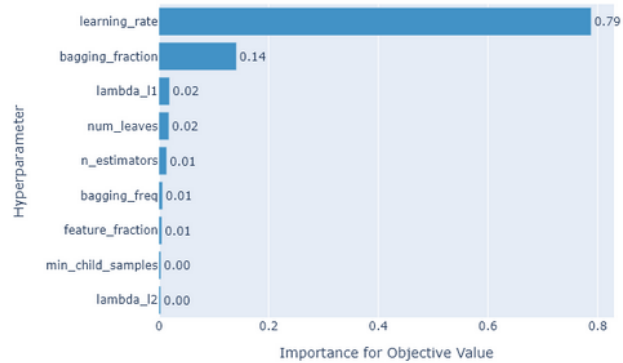


Figure 13: LGBMClassifier Hyperparameter Importances

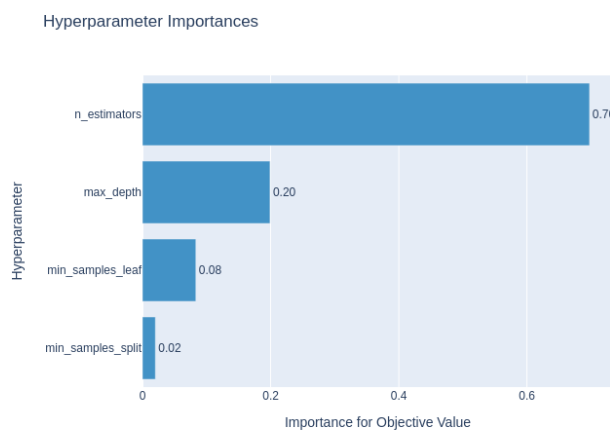


Figure 14: Random Forest Hyperparameter Importances

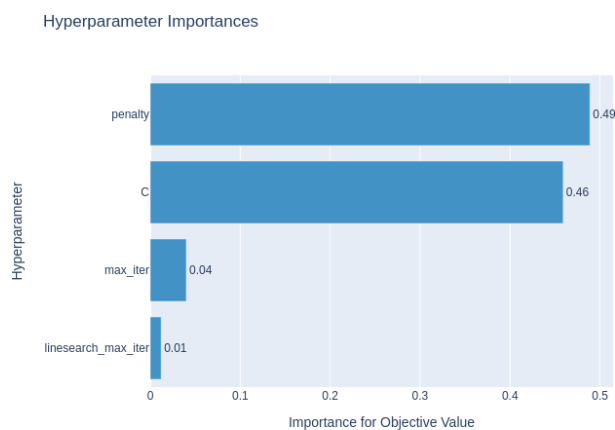


Figure 15: Stacking Model Hyperparameter Importances

Appendix - C: Base Model Correlations

Table 18: Correlations of base models used in stacking

	XGBClassifier	LGBMClassifier	Logistic Regression	Random Forest Classifier
XGBClassifier	-	0.996636	0.983362	0.982002
LGBMClassifier	0.996636	-	0.984408	0.982881
Logistic Regression	0.983362	0.984408	-	0.976551
Random Forest Classifier	0.982002	0.982881	0.976551	-